



T H E D B A D M I N
P R A C T I T I O N E R S E R I E S

Patroni Internals

A Practitioner's Theory Guide

*How every component works, why each one exists,
and what the cluster is actually guaranteeing you*

For PostgreSQL database administrators,
site reliability engineers and platform teams

Verified against Patroni 4.1.5

First Edition · 2026

thedbadmin.com

Patroni Internals: A Practitioner's Theory Guide

TheDBAdmin Practitioner Series · First Edition, 2026

Published by TheDBAdmin, Bengaluru, India · thedbadmin.com

Copyright

© 2026 TheDBAdmin. All rights reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means, including photocopying, recording or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

This volume is produced for use in TheDBAdmin corporate training programmes and for individual practitioner study. Licensed classroom use is permitted under the terms of the applicable training agreement.

Trademarks

PostgreSQL is a registered trademark of the PostgreSQL Community Association of Canada. Patroni is developed and maintained by its contributors and is distributed under the MIT licence; the project was originated at Zalando SE and Compose. etcd is a project of the Cloud Native Computing Foundation. Consul, and HashiCorp are trademarks of HashiCorp, Inc. Apache ZooKeeper is a trademark of the Apache Software Foundation. Kubernetes is a registered trademark of The Linux Foundation. All trademarks are the property of their respective owners and are used here for identification only.

Accuracy and disclaimer

Every command, parameter default, key path and behavioural claim in this book was verified against the official Patroni 4.1.5 documentation and the PostgreSQL 16 and 17 documentation during preparation. Software evolves; readers must confirm defaults against the versions they operate before acting on them in production.

The laboratory exercises in Part V are deliberately destructive and are intended solely for disposable test environments. The publisher and author accept no liability for loss or damage arising from the application of any procedure in this book to a production system.

Feedback and errata

Corrections, classroom feedback and requests for the companion lab environment may be directed to thedbadmin.com.

Contents

Preface.....	2
Who this book is for · Prerequisites · Learning objectives.....	2
PART I — THE PROBLEM AND THE CONCEPT.....	4
1. Why PostgreSQL Cannot Fail Over By Itself.....	4
2. What Patroni Is.....	5
PART II — THE COMPONENT MODEL.....	7
3. Anatomy of a Patroni Cluster.....	7
4. The Distributed Configuration Store.....	11
5. Consensus, Leases and the Arithmetic of Time.....	13
PART III — BEHAVIOUR UNDER NORMAL AND FAILED CONDITIONS.....	16
6. The Heartbeat Loop.....	16
7. Leader Election and Failover.....	18
8. Preventing Split-Brain.....	20
9. Replication Modes and the Durability Spectrum.....	21
10. Bootstrap, Replica Creation and Rejoin.....	23
PART IV — JUDGEMENT: PRACTICE, LIMITS AND ALTERNATIVES.....	25
11. Best Practices.....	25
12. Trade-offs and When Patroni Is the Wrong Tool.....	26
13. Common Mistakes.....	27
14. Comparative Analysis.....	27
15. Advanced Extensions.....	28
PART V — REINFORCEMENT.....	29
16. Laboratory Exercises.....	29
17. Assessment.....	31
Answer Key.....	32
Appendix A — Parameter Quick Reference.....	34
Appendix B — References.....	35

FIGURES

Figure 1 Component model of a three-node Patroni cluster.....	7
Figure 2 The DCS key hierarchy.....	11
Figure 3 One iteration of the heartbeat loop.....	16
Figure 4 Unplanned failover timeline.....	18
Figure 5 The four guards against split-brain.....	20
Figure 6 The four replication modes.....	21

Preface

Who this book is for

This book is written for database administrators, site reliability engineers and platform engineers who already operate PostgreSQL in production and now need to reason about Patroni rather than merely run it. It assumes you have promoted a standby by hand at least once, that you know what a WAL segment is, and that you have felt the particular anxiety of a primary that will not come back.

It is deliberately a theory guide. It contains configuration fragments and commands, but they appear as evidence for an explanation, not as a deployment recipe. If your goal is to stand up a three-node cluster this afternoon, the installation documentation will serve you faster. If your goal is to be the person in the incident bridge who can say why the cluster did what it did, this is the book.

Prerequisites

- Working knowledge of PostgreSQL streaming replication: WAL, LSN, timelines, replication slots and hot standby.
- **Comfort with a Linux shell** — systemd units, journal inspection, and reading a YAML file without flinching.
- Basic familiarity with distributed systems vocabulary: quorum, partition, consensus. No prior Raft knowledge is assumed; Chapter 5 builds it from first principles.
- No prior Patroni experience is required, although operators who have used it will find the mechanics they have observed finally explained.

Learning objectives

By the end of this book you should be able to do the following without consulting the source:

1. Name every component of a Patroni cluster and state precisely which failure each one is defending against.
2. Draw the key hierarchy a Patroni cluster maintains in its Distributed Configuration Store, and explain which keys expire and why.
3. Explain the heartbeat loop as a state machine, and predict what a node will do on the next iteration given a described cluster state.
4. Derive the worst-case failover window from the timing parameters, and justify the constraint that relates them.
5. Enumerate the mechanisms that prevent two primaries from existing simultaneously, and identify which of them is active in a given configuration.
6. Choose between asynchronous, synchronous, strict and quorum replication modes from a stated durability requirement, and articulate the availability cost of each.
7. Recognise the failure classes Patroni does not address, and name the tool that does.

How this book is organised

The book follows a deliberate teaching sequence. Part I frames the problem before naming the tool, because Patroni is unintelligible until you feel the gap it fills. Part II builds the static picture: what the pieces are and how they connect. Part III sets that picture in motion, tracing the loop, the election and the guards that keep the cluster honest. Part IV steps back to judgement — best practice, trade-off, failure mode and comparison against the alternatives. Part V is reinforcement: five laboratory exercises graded from foundational to advanced, and a twenty-question assessment with a worked answer key.

Chapters are self-contained enough to be lifted into a classroom module, but the argument is cumulative. A reader who skips Chapter 5 will find the timing arithmetic in Chapter 7 arbitrary rather than inevitable.

Notation and conventions

Convention	Meaning
Monospace inline	A configuration key, a DCS path, a command or a PostgreSQL parameter.
Code panel with gold bar	A validated command, configuration fragment or output sample. Safe to adapt.
Gold callout	A definition or a rule worth memorising.
Red callout	A trap, a destructive operation or a common misreading.
Figure n	A numbered diagram. All figures are original to this volume.
/leader, /config ...	Keys relative to the cluster namespace, i.e. /service/<scope>/leader.

VERSION AND VERIFICATION

Every parameter default, endpoint name, key path and behavioural claim in this book was checked against the Patroni 4.1.5 documentation and the PostgreSQL 16 and 17 documentation at the time of writing. Where a default is quoted, it is the documented default and not a site convention.

Patroni is actively developed. Before relying on a specific default in production, confirm it against the version you are running with `patronictl show-config`.

PART I

The Problem and the Concept

1. Why PostgreSQL Cannot Fail Over By Itself

1.1 What PostgreSQL gives you

PostgreSQL ships with a remarkably complete replication toolkit. Physical streaming replication moves write-ahead log records from a primary to any number of standbys with sub-second latency. Replication slots guarantee the primary retains WAL a standby still needs. Cascading replication lets a standby feed further standbys. Synchronous commit lets you demand that a transaction reach a standby's disk before the client is told it committed. Hot standby lets those standbys serve read traffic while they replay.

Every one of those mechanisms is production-grade and has been for over a decade. The toolkit is not the problem.

1.2 What PostgreSQL withholds

What PostgreSQL does not ship is a decision. It will replicate faithfully, but it will never decide that the primary is dead, never decide which standby should take over, and never promote anything on its own initiative. Promotion is an explicit administrative act, invoked by an operator or a script.

This is not an oversight. A single PostgreSQL instance has no vantage point from which to make that decision safely. A standby that cannot reach the primary has learned exactly one fact: it cannot reach the primary. It has learned nothing about whether the primary is dead, whether the primary is alive and serving other clients perfectly well, or whether the network between them has failed while both remain healthy. From inside one process, those three worlds are indistinguishable.

THE FUNDAMENTAL RESULT

In an asynchronous network, no algorithm can distinguish a crashed process from a slow one. This is the **impossibility of perfect failure detection**, and it is not an engineering limitation to be solved by better code — it is a property of the system model. Every high-availability tool, Patroni included, works around it rather than solving it: it substitutes a **timeout** for a truth, and then builds machinery to make acting on that timeout safe.

1.3 Split-brain: the failure that costs you the data

Suppose you write the obvious script. It runs on each standby, pings the primary every few seconds, and promotes the local instance after five consecutive failures. Deploy it to a three-node cluster and consider a network partition that isolates the primary from both standbys, while all three machines remain running and reachable by some subset of your application servers.

Both standbys detect failure. Both promote. The old primary, which never lost power, continues accepting writes from whichever clients can still reach it. You now have three PostgreSQL instances that each believe they are authoritative, each accumulating transactions on a diverging timeline. This is split-brain.

The damage is asymmetric and permanent. An outage is recoverable: the service returns and the data is intact. Split-brain destroys data by construction, because reconciling two divergent write histories is not a

database operation. There is no merge. Someone chooses which timeline to keep, and the transactions on the other one are gone, whether or not a customer was told they had committed.

DESIGN CONSEQUENCE

Because split-brain is strictly worse than downtime, every correct HA design biases towards refusing to promote when it is uncertain. Patroni would rather leave your cluster read-only than risk a second primary. Operators who find Patroni "too conservative" during an incident are usually observing this bias working exactly as intended.

1.4 Why the script is never enough

The failure of the naive script is not that it was written carelessly. Its failure is structural: it makes an uncoordinated decision using local information. Fix it by adding a peer check and it still fails, because the two standbys can reach each other inside the partition and will agree with each other while being collectively wrong. Fix that by requiring a majority and you have begun, badly, to reimplement distributed consensus.

This is the insight that gives Patroni its shape. Automatic failover is not a monitoring problem that happens to touch a database. It is a distributed agreement problem with a database attached. The correct architecture therefore does not try to make PostgreSQL smarter. It places a co-ordinating agent beside each PostgreSQL instance, and gives those agents a shared, strongly consistent place to record exactly one fact: who is currently allowed to be primary.

2. What Patroni Is

2.1 A template, not a product

Patroni describes itself as a template for building highly available PostgreSQL clusters. The wording is precise and worth taking seriously. Patroni is a Python daemon that supplies the co-ordination logic and delegates every other concern: consensus to an external store, replication to PostgreSQL, connection routing to a proxy you choose, and fencing to the operating system. It is opinionated about the decision and agnostic about nearly everything else.

Practically, this means adopting Patroni is not a migration. The PostgreSQL binaries, the data directory, the WAL, the backup tooling and the monitoring you already run are unchanged. What changes is who starts PostgreSQL, who writes the recovery configuration, and who decides which node is primary. Those three responsibilities move to Patroni, and they move completely — an operator who starts PostgreSQL by hand underneath a running Patroni has created a fault, not a workaround.

2.2 Control plane and data plane

The single most useful mental model is the separation between the control plane and the data plane, and it is the organising idea of Figure 1.

The data plane is PostgreSQL streaming replication. WAL flows from primary to standbys over the ordinary replication protocol on port 5432. Patroni is not in this path. It does not proxy, inspect, buffer or transform a single byte of your data. If every Patroni agent in the cluster were killed simultaneously, replication would continue undisturbed until something else needed to change.

The control plane is everything Patroni does: observing PostgreSQL, publishing state to the Distributed Configuration Store, competing for the leader lock, and issuing promote or demote instructions locally. It runs over the DCS protocol and over Patroni's own REST API on port 8008.

WHY THE SEPARATION MATTERS OPERATIONALLY

Patroni is not a performance-critical component and does not sit in the query path, so it cannot add query latency. It follows that a Patroni problem is almost never diagnosed by looking at query performance, and a query performance problem is almost never fixed by touching Patroni. Diagnose in the plane where the symptom lives.

2.3 The three responsibilities

Everything Patroni does reduces to three duties, and it is worth being able to recite them:

- 8. Manage the local PostgreSQL lifecycle.** Patroni starts, stops, restarts, promotes and demotes the instance on its own node. It also writes the parts of the configuration that must be consistent across the cluster, so that any node is ready to be primary at any moment.
- 9. Publish and consume cluster state.** Each agent writes what it knows about itself into the DCS and reads what the others have written. No agent trusts its own memory of the cluster across iterations; each cycle re-derives the picture from the shared store.
- 10. Contend for, hold and surrender the leader lock.** Exactly one node may hold a short-lived lease in the DCS. Holding it is what authorises a node to run as primary. Losing it is what obliges a node to stop being one.

That third duty is the heart of the design. Patroni does not decide who the primary is by voting among agents. It converts the question into a race for a single atomically-created, automatically-expiring key, and lets the consensus store adjudicate. The correctness of the whole system rests on the store's guarantee that at most one contender can create that key.

PART II

The Component Model

3. Anatomy of a Patroni Cluster

A minimal production Patroni deployment involves seven distinct kinds of component. Reading them as a list obscures the structure; reading them as three tiers makes it obvious. Figure 1 shows the canonical arrangement.

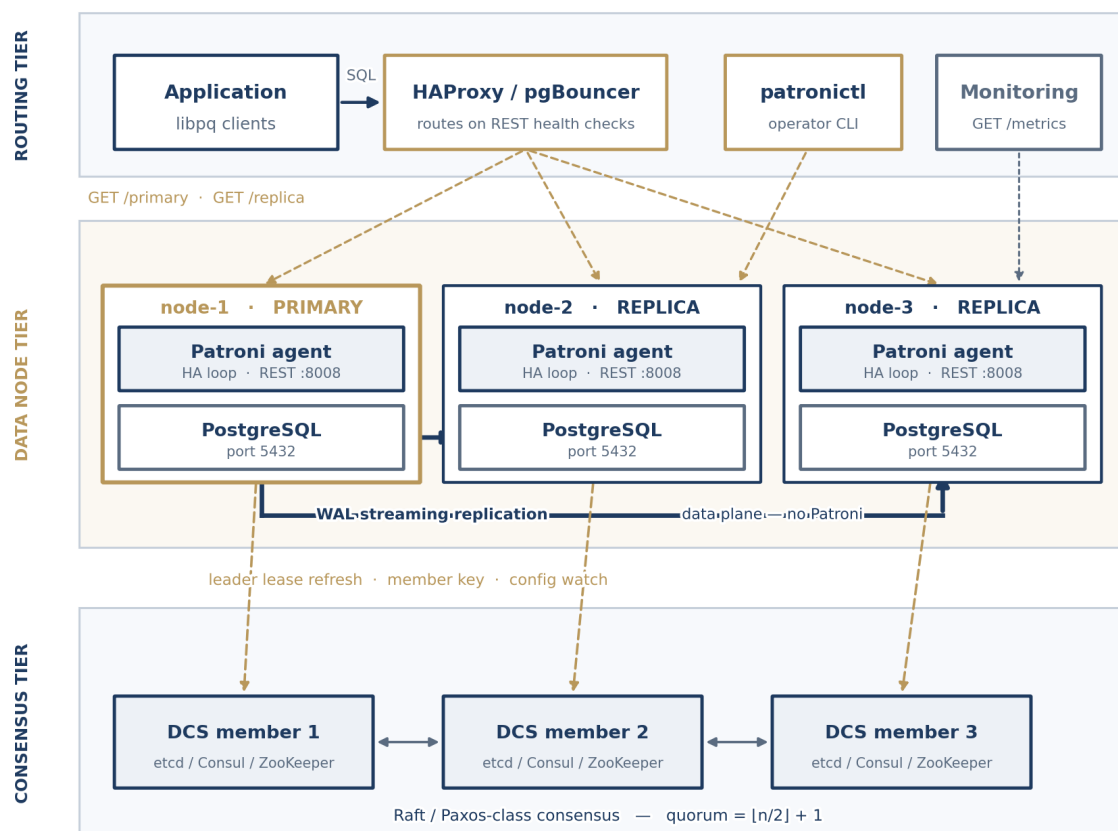


Figure 1 — Component model of a three-node Patroni cluster. Gold dashed lines are control-plane traffic; the solid navy path is data-plane WAL replication, which Patroni never touches.

3.1 Component inventory

Component	Runs where	Responsibility	Failure it defends against
Patroni agent	One per data node	Runs the HA loop; manages local PostgreSQL; contends for the leader lock	Unco-ordinated or absent promotion decisions
PostgreSQL	One per data node	Stores and replicates the data	— (this is the thing being protected)
DCS member	Odd-numbered cluster, ideally separate hosts	Provides atomic, expiring, strongly consistent keys	Two nodes both believing they hold the lock

Component	Runs where	Responsibility	Failure it defends against
REST API	Inside each agent, port 8008	Exposes node state; used in the leader race, by proxies and by operators	Routing traffic to a demoted or lagging node
Watchdog	Kernel device on each data node	Resets the host if the agent stops petting it	A hung agent leaving PostgreSQL writable
patronictl	Operator workstation or any node	Controlled switchover, restart, reinit, configuration edit	Human intervention that bypasses the lock
Connection router	HAProxy, pgBouncer, VIP or DNS	Sends writes to whoever is currently primary	Clients pinned to a former primary

3.2 The Patroni agent

The agent is a single Python process, conventionally managed by systemd, running on the same host as the PostgreSQL instance it governs. Co-location is not incidental: the agent must be able to inspect the data directory, read the control file, and signal the postmaster. An agent managing a remote PostgreSQL would be unable to fence it, and fencing is half the point.

Internally the agent is a loop rather than an event handler. It wakes on a fixed interval, gathers facts, consults the DCS, decides on one action, performs it, and sleeps. Chapter 6 traces that loop in detail. The important structural property is that the agent is stateless between iterations: it holds no durable opinion about the cluster that it would need to reconcile after a restart.

3.3 The PostgreSQL instance

Patroni takes ownership of a defined slice of the PostgreSQL configuration. Parameters that must agree across the cluster — because a standby cannot start with smaller shared-memory structures than its primary — are removed from local control and managed only through the shared dynamic configuration.

Parameter	Documented default	Documented minimum
max_connections	100	25
max_locks_per_transaction	64	32
max_worker_processes	8	2
max_prepared_transactions	0	0
max_wal_senders	10	3
max_replication_slots	10	4
wal_keep_size	128MB	16MB
wal_level	hot_standby	hot_standby / replica / logical

Patroni enforces these by writing them into postgresql.conf and additionally passing them on the postmaster command line, which gives them precedence over ALTER SYSTEM. The original operator-managed configuration is preserved: on first start Patroni renames postgresql.conf to postgresql.base.conf and includes it, so hand-maintained settings survive.

A RESTART ORDERING RULE WORTH MEMORISING

When **increasing** any shared-memory parameter, restart all standbys first and the primary last. When **decreasing** one, restart the primary first and the standbys after. Reversing this leaves standbys unable to start, because PostgreSQL refuses to run a standby whose shared-memory structures are smaller than those recorded in the primary's control file.

3.4 The Distributed Configuration Store

The DCS is where the cluster's single source of truth lives. Patroni supports etcd (v2 and v3 protocols), Consul, ZooKeeper, the Kubernetes API, AWS-hosted equivalents, and a built-in Raft implementation that needs no external service. Chapter 4 covers the store in detail; here it is enough to state the contract Patroni requires of it.

- **Atomic create.** A key can be created only if it does not already exist, and exactly one of several simultaneous creators succeeds.
- **Compare-and-swap update.** A key can be updated only if its current value or version matches what the writer expects.
- **Automatic expiry.** A key can carry a time-to-live and vanish without anyone deleting it — including when the writer has died.
- **Linearizable reads.** A successful read reflects all previously acknowledged writes. Stale reads would let a demoted node believe it is still leader.

Any store providing those four primitives can back Patroni. Everything else — the wire protocol, the deployment shape, the operational tooling — is a matter of taste and existing infrastructure.

3.5 The REST API

Each agent exposes an HTTP interface on port 8008. It is easy to mistake this for a monitoring convenience. It is not: it is load-bearing. Patroni uses it during the leader race to query peers for their WAL positions, and the DCS failsafe mechanism of Chapter 8 uses it to establish liveness when the DCS itself is unreachable.

Endpoint	Returns 200 when	Typical consumer
GET / · /primary · /read-write	Node is primary and holds the leader lock	HAProxy write backend
GET /leader	Node holds the lock, regardless of primary or standby-leader role	Multi-datacentre routing
GET /standby-leader	Node leads a standby cluster	Cascading topologies
GET /replica	State is running, role is replica, noloadbalance not set	HAProxy read backend
GET /replica?lag=10MB	As above, and lag is under the stated bound	Latency-sensitive read pools
GET /sync · /quorum	Node is a synchronous, or quorum-voting, standby	Zero-loss read routing
GET /health	PostgreSQL is up and running	Coarse liveness
GET /liveness	Heartbeat loop ran within ttl (2×ttl on a replica)	Kubernetes livenessProbe
GET /readiness	Node is leader, or is replicating within the lag bound	Kubernetes readinessProbe
GET /patroni · /cluster	Always; returns node or cluster state as JSON	Monitoring, and Patroni's own leader race

Endpoint	Returns 200 when	Typical consumer
GET /metrics	Always; Prometheus exposition format	Prometheus scrape
POST /switchover /failover	Operation accepted (202 if scheduled)	patronictl, automation

A ROUTING RULE THAT PREVENTS A REAL OUTAGE

Route your write traffic at `/primary`, never at `/health`. A demoted former primary still passes `/health` — PostgreSQL genuinely is up and running — and a proxy checking liveness rather than role will happily send writes to a read-only node, producing a flood of errors that looks like a database fault but is a configuration fault.

3.6 The watchdog

The watchdog is the component operators most often skip and most often regret skipping. It answers a question the rest of the design cannot: what if the Patroni agent itself stops running while PostgreSQL keeps serving writes?

Every other guard assumes the agent is alive to enforce it. A process suspended by memory pressure, blocked on I/O, or stopped by an errant SIGSTOP will not demote its PostgreSQL, because demotion is something the agent does. Meanwhile the lease expires, a standby is promoted, and the frozen node's PostgreSQL is still accepting connections.

A watchdog closes this by moving enforcement into the kernel. Patroni opens the watchdog device and must write to it periodically. If those writes stop, the kernel resets the machine — killing PostgreSQL without needing any co-operation from the process that failed. On Linux this is normally the software watchdog:

```
# Load the kernel software watchdog and hand the device to Patroni
modprobe softdog
chown postgres /dev/watchdog

# For lab use only – prevents an actual reboot when the watchdog fires
modprobe softdog soft_noboot=1
```

The timing is derived rather than configured directly. Patroni sets the watchdog to expire before the leader lease does, reserving a `safety_margin` between the moment the lease is confirmed and the moment the `keepalive` is sent. With defaults, the documented guarantee is that PostgreSQL has at least `ttr - safety_margin - loop_wait - retry_timeout` seconds to terminate all client connections once the DCS becomes unavailable — five seconds under stock settings. Setting `safety_margin` to `-1` sets the watchdog timeout to half the TTL, which makes the guarantee unconditional at the cost of a tighter margin.

3.7 patronictl

The operator CLI is the supported way for a human to intervene. Its significance is architectural rather than convenience: `patronictl` performs its work through the same DCS keys and REST endpoints the agents use, so an operator action participates in the protocol instead of subverting it. A switchover requested through `patronictl` is recorded, arbitrated and executed by the cluster; the same outcome achieved with `pg_ctl promote` is an unrecorded divergence that Patroni will detect and attempt to correct.

3.8 The connection router

Patroni deliberately does not move client connections. When a failover completes, the cluster has a new primary and every existing client is still connected to the old one. Something outside Patroni must close that gap — commonly HAProxy polling the REST endpoints, but equally pgBouncer, a floating virtual IP with Keepalived, service-based routing in Kubernetes, or multi-host connection strings in the client driver.

THE MOST COMMON ARCHITECTURAL OMISSION

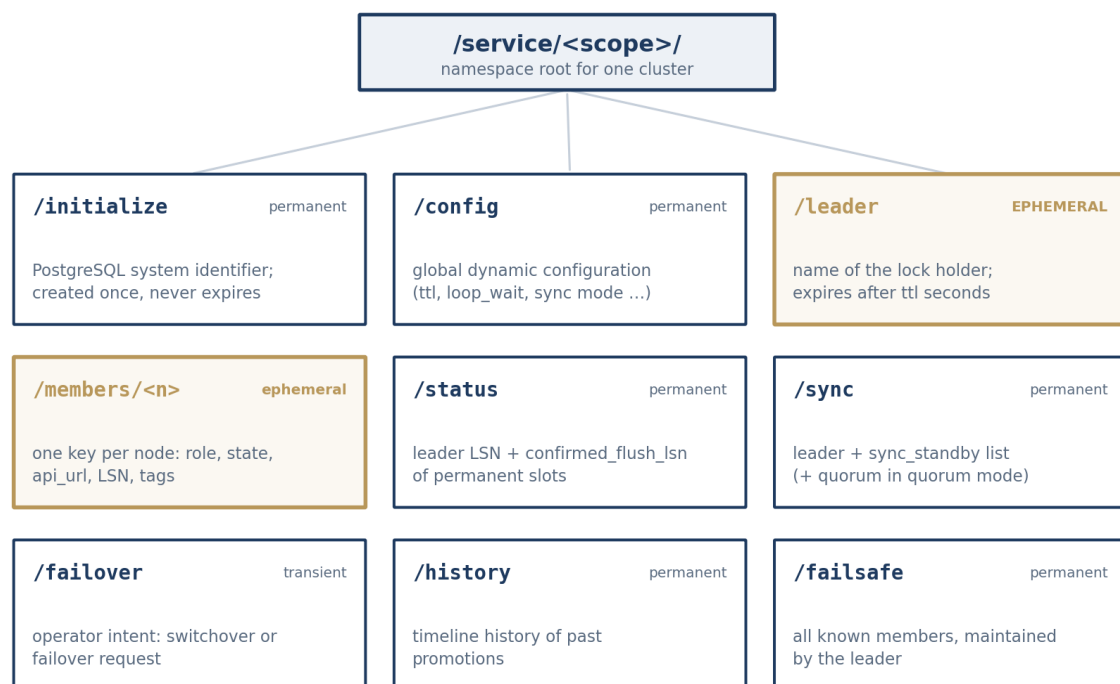
A single HAProxy in front of a three-node Patroni cluster reintroduces exactly the single point of failure the cluster was built to eliminate. If the router is a single instance, cluster availability is bounded by router availability, and the database tier's redundancy is decorative. Deploy the router redundantly — a pair with a shared VIP, or one instance per application host.

4. The Distributed Configuration Store

If Patroni has a single conceptual centre, it is this: the cluster's authoritative state is not held in any PostgreSQL instance and not held in any agent's memory. It is held as a small set of keys in an external, strongly consistent store, and every decision any agent makes is derived from reading them.

4.1 The key hierarchy

All keys live under a namespace derived from two configuration values: the namespace (default `/service/`) and the scope, which names the cluster. A cluster with scope `batman` therefore keeps its state under `/service/batman/`. Figure 2 shows the complete set.



Only `/leader` and `/members` carry a TTL. Every write that must be exclusive uses Compare-And-Swap.

Figure 2 — The key hierarchy Patroni maintains for one cluster. Only `/leader` and the individual member keys carry a time-to-live.

Key	Written by	Lifetime	Purpose
<code>/initialize</code>	First node to bootstrap	Permanent	Holds the PostgreSQL system identifier. Its existence proves the

Key	Written by	Lifetime	Purpose
			cluster was already created and stops a second node from initialising over it.
/config	Operator, via patronictl or REST	Permanent	Global dynamic configuration: timing parameters, replication mode, managed PostgreSQL parameters.
/leader	The current leader	TTL = ttl seconds	The lock. Contains the holder's name. Holding it authorises running as primary.
/members/<name>	Each agent, about itself	TTL = ttl seconds	Role, state, API URL, WAL position, timeline and tags for one node.
/status	The leader	Permanent	Leader WAL position and confirmed flush positions of permanent slots, so replicas can compute lag cheaply.
/sync	The leader	Permanent	Current synchronous standby set, and the quorum size in quorum mode.
/failover	Operator	Consumed on action	A requested switchover or failover, optionally scheduled.
/history	Promoting node	Permanent	Timeline history of past promotions, mirroring the WAL history file.
/failsafe	The leader	Permanent	The full member list, used by failsafe mode to verify a partition.

4.2 Two primitives do all the work

Only two atomic operations underpin the entire protocol, and both are supplied by the store rather than implemented by Patroni.

Compare-and-create is used for acquisition. A candidate asks the store to create /leader only if it does not exist. If several candidates ask simultaneously, the store serialises them and returns success to exactly one. The losers learn immediately that they lost, and there is never an interval in which two nodes both believe they succeeded.

Compare-and-swap is used for retention. The holder asks the store to update /leader only if its current value is still the holder's own name. This is subtly but critically different from an unconditional write. A node that was leader, was partitioned, had its key expire, and now returns will find its compare-and-swap rejected — because the key is either absent or now names someone else. It cannot silently reclaim leadership by writing over the truth.

WHY THE LEASE IS A LEASE AND NOT A HEARTBEAT

A heartbeat requires an observer to notice absence and act. A lease requires nobody: the key removes itself. This inverts the failure mode in a way that matters. If Patroni's monitoring is broken, a heartbeat scheme fails to detect a dead primary and the cluster stalls; a lease scheme still expires the key, because expiry is the **store's** responsibility. The safety property survives the failure of the thing watching it.

4.3 Choosing a store

Store	Consensus	Operational weight	Best suited to
etcd (v3)	Raft	Light — single static binary, three nodes	The default recommendation. Purpose-built for exactly this workload.
Consul	Raft, plus Serf gossip	Heavier — server and client	Sites already running Consul for service discovery, and

Store	Consensus	Operational weight	Best suited to
		agent tiers	multi-datacentre estates.
ZooKeeper	ZAB	Heavy — JVM, ensemble tuning	Estates with existing ZooKeeper operational expertise, often alongside Kafka or Hadoop.
Kubernetes API	Raft, via the cluster's own etcd	None additional	Patroni running inside Kubernetes. Note that API server contention makes lock-update failures more frequent, which is precisely why failsafe mode exists.
Patroni Raft (pysyncobj)	Raft, embedded in the agents	None additional	Small clusters and labs where a separate store is disproportionate. Fewer operational tools and less battle-tested at scale.

RECOMMENDATION

For a greenfield on-premises or cloud-VM deployment, use etcd v3 on three dedicated small hosts. It is the most widely exercised path, the failure modes are the best documented, and the operational surface is a single binary. Choose Consul or ZooKeeper when you already run one and have the expertise; choose the Kubernetes API when you are already inside Kubernetes; choose embedded Raft only when the alternative is not deploying HA at all.

4.4 Sizing and placement

The DCS is a quorum system: it tolerates the loss of fewer than half its members. Three members tolerate one failure; five tolerate two. Even-numbered clusters are strictly worse than the odd number below them — four members still tolerate only one failure, while adding a member that can itself fail. Always deploy an odd count.

Placement matters as much as count. Three DCS members in one rack, or one availability zone, give quorum arithmetic that is technically correct and operationally meaningless: the event that takes out the rack takes out the cluster's ability to elect anything. Distribute members across the failure domains you actually intend to survive.

There is one further placement question that divides practitioners: whether to co-locate DCS members with the PostgreSQL nodes. Co-location is cheaper and keeps the round trip local. Separation avoids a class of correlated failure in which memory or I/O pressure from PostgreSQL delays DCS responses, causing the lease update to time out and the primary to demote itself — a self-inflicted failover under load. Where budget permits, separate them.

5. Consensus, Leases and the Arithmetic of Time

This chapter supplies the theory that makes the rest of the book inevitable rather than arbitrary. It is short, and skipping it is the main reason operators find Patroni's timing parameters mysterious.

5.1 Raft in one page

Raft is a consensus algorithm designed to be teachable, and it backs both etcd and Consul. It solves one problem: getting a set of machines to agree on an ordered log of operations even when some of them fail.

- **Terms.** Time is divided into numbered terms. Each term has at most one leader. A term number that is higher always wins over a lower one, which gives the system a total order on leadership.

- **Election.** A member that has not heard from a leader within its election timeout increments the term and solicits votes. A member grants at most one vote per term. A candidate that collects a majority becomes leader. Because two majorities of the same set must overlap in at least one member, and that member votes once, two leaders cannot be elected in the same term.
- **Log replication.** The leader appends entries and replicates them. An entry is committed once a majority has stored it, and only committed entries are applied. A minority partition can append nothing, because it can never reach a majority.

The majority-overlap argument is the whole safety story, and it is worth being able to state in one sentence: any two majorities of the same set share at least one member, and that shared member's single vote per term makes two simultaneous leaders impossible.

THE DISTINCTION OPERATORS MOST OFTEN GET WRONG

The Raft leader inside etcd and the PostgreSQL primary in your Patroni cluster are **completely unrelated**. etcd elects a Raft leader to order its own writes; Patroni elects a PostgreSQL primary by racing for a key that etcd happens to store. An etcd leader election is a routine internal event that occurs without any effect on your database. Alarming on it produces noise, not signal.

5.2 Why Patroni does not run its own consensus

Patroni could have implemented Raft among its agents. It deliberately does not, for two reasons worth understanding.

The first is correctness economics. Consensus implementations are notoriously difficult to get right, and the bugs surface only under partition, clock skew and message reordering — precisely the conditions under which you most need them to be correct. Delegating to etcd or ZooKeeper means inheriting implementations that have been exercised by thousands of operators for years.

The second is separation of concerns. The agents must be free to restart, upgrade and crash without disturbing the record of who is leader. If that record lived inside the agent quorum, a rolling upgrade of Patroni would be a consensus membership change. Keeping state outside the agents makes them disposable, which is exactly what you want from the component most likely to be restarted.

5.3 Lease arithmetic

Three parameters govern the timing of the entire system. Their defaults are not arbitrary, and the constraint relating them is enforced by Patroni rather than merely advised.

Parameter	Default	Minimum	Meaning
ttd	30 s	20 s	Lifetime of the leader lock. The upper bound on how long a dead primary's lock survives, and hence on failure detection.
loop_wait	10 s	1 s	Sleep between heartbeat-loop iterations. The granularity at which every node re-evaluates the world.
retry_timeout	10 s	3 s	How long DCS and PostgreSQL operations are retried before being treated as failed. A blip shorter than this does not demote the leader.
maximum_lag_on_failover	1048576 (1 MB)	—	How far behind a candidate may be and still be eligible for automatic promotion.

THE CONSTRAINT

$$\text{ttl} \geq \text{loop_wait} + 2 \times \text{retry_timeout}$$

With defaults: $30 \geq 10 + 20$, which holds exactly. The reasoning is that a leader must have at least one full opportunity to refresh its lease, including a complete round of retries, before the lease expires. Violate the constraint and you create a cluster that fails over during ordinary network jitter, because the leader runs out of lease while still legitimately retrying.

Tuning follows directly. Lowering ttl shortens detection and therefore downtime, but narrows the margin for a slow DCS response, making spurious failovers more likely. Raising it makes the cluster more tolerant of jitter at the cost of a longer outage when a primary genuinely dies. Ten seconds is generally the practical floor on shared or virtualised infrastructure; going lower demands dedicated hardware and a local DCS.

The documented worst case for a primary failure is $\text{loop_wait} + \text{primary_start_timeout} + \text{loop_wait}$, because Patroni first allows a crashed primary the chance to recover in place before initiating failover. Setting `primary_start_timeout` to zero removes that grace period and fails over as soon as a crash is detected — faster, and with a correspondingly higher risk of losing transactions that a recovering primary would have retained.

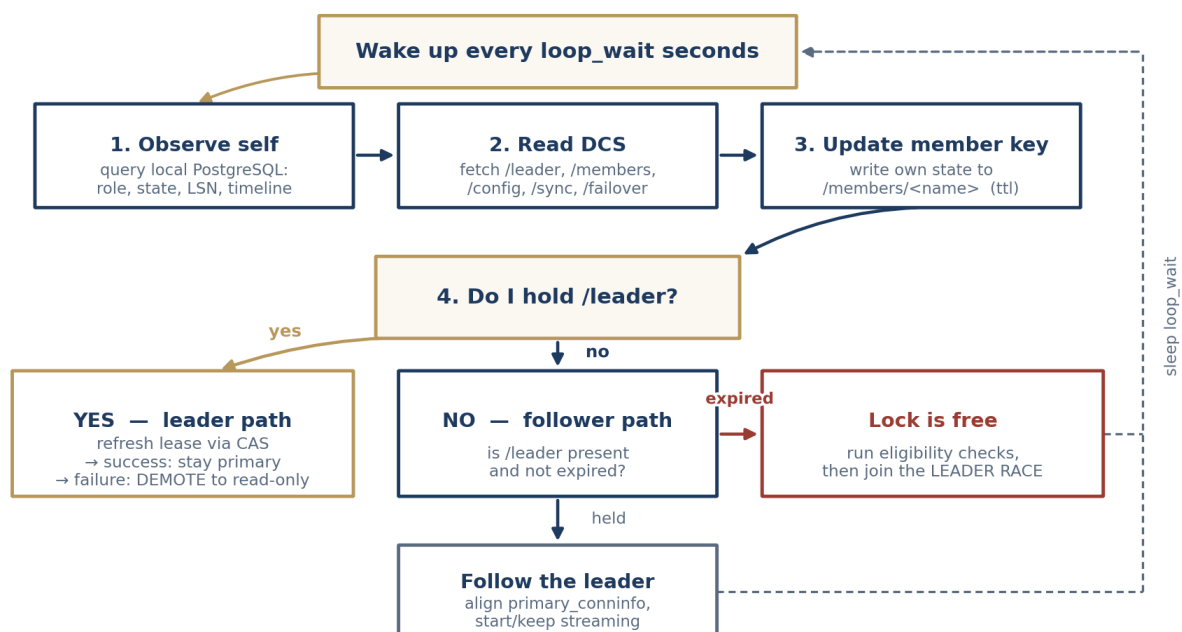
PART III

Behaviour Under Normal and Failed Conditions

6. The Heartbeat Loop

Patroni's behaviour is entirely produced by one loop running independently on every node. There is no central scheduler and no orchestrator. Each agent runs the same code, reads the same shared state, and arrives at a decision about itself alone. The cluster's co-ordinated behaviour is emergent, not directed.

6.1 One iteration



The loop is idempotent: every iteration re-derives the correct action from current DCS state, never from remembered state.

Figure 3 — One iteration of the heartbeat loop. Every node runs this independently, every `loop_wait` seconds.

Each iteration proceeds in the same order:

11. **Observe self.** Query the local PostgreSQL for its role, running state, WAL position and timeline. If PostgreSQL is not running, determine why — crashed, stopped deliberately, or never bootstrapped.
12. **Read the DCS.** Fetch the cluster view: `/leader`, all `/members`, `/config`, `/sync` and any pending `/failover` request. This read is authoritative and replaces anything the agent believed from the previous iteration.
13. **Publish self.** Write the observed local state into `/members/<name>` with a TTL. This is how the node advertises its WAL position to potential future rivals, and how its disappearance becomes visible when it dies.
14. **Decide and act.** Determine whether this node holds the leader lock, and follow the corresponding branch. Exactly one action is taken per iteration.
15. **Sleep.** Wait `loop_wait` seconds and repeat.

6.2 The leader branch

A node holding the lock attempts a compare-and-swap refresh of `/leader`, asserting that the current value is still its own name. Success renews the lease for another `tli` seconds and the node continues as primary.

Failure is the interesting case, and Patroni's response is deliberately severe. If the refresh cannot be completed within `retry_timeout`, the agent demotes PostgreSQL to read-only — without knowing whether the DCS is down or the network has partitioned. It assumes the worse of the two possibilities, because in a partition another node may already be promoting, and a demotion that turns out to have been unnecessary costs an availability blip while a missed one costs data.

6.3 The follower branch

A node not holding the lock asks a single question: does a valid `/leader` key exist? If it does, the node ensures it is following that leader — checking that `primary_conninfo` points at the right host and that streaming is established, and correcting the configuration if not.

If the key is absent or expired, the lock is free and the node evaluates whether it may compete. Eligibility is checked before the race, not after, and Chapter 7 enumerates the tests.

6.4 Why the loop is idempotent

The loop never carries a decision forward. Every iteration re-derives the correct action from the current contents of the DCS, which is why a Patroni agent can be killed at any point and restarted without any recovery procedure. There is no in-flight transaction to roll back and no local state file to reconcile — on its next iteration, the restarted agent simply reads the world and acts on it.

This property is also why pausing the cluster is safe. In pause mode the loop continues to run and continues to report, but takes no corrective action, which is exactly what you want during maintenance that would otherwise look like a failure.

6.5 Reading node state

The state a node reports through `/patroni` and `/metrics` is a fixed enumeration. The numeric values are guaranteed stable across versions so that monitoring systems can alert on them:

Value	State	Meaning
0 – 1	<code>initdb</code> , <code>initdb_failed</code>	Creating a brand-new cluster, or having failed to
2 – 3	<code>custom_bootstrap</code> , <code>..._failed</code>	Running an operator-supplied bootstrap method
4	<code>creating_replica</code>	Cloning from the primary or from a backup
5	<code>running</code>	Normal operation
6 – 8	<code>starting</code> , <code>bootstrap_starting</code> , <code>start_failed</code>	Coming up, or unable to
9 – 10	<code>restarting</code> , <code>restart_failed</code>	Restart in progress or failed
11 – 13	<code>stopping</code> , <code>stopped</code> , <code>stop_failed</code>	Shutdown in progress, complete, or stuck
14	<code>crashed</code>	PostgreSQL terminated abnormally

7. Leader Election and Failover

7.1 The timeline of an unplanned failover

Consider a three-node cluster with default timing whose primary loses power. Figure 4 traces what follows.

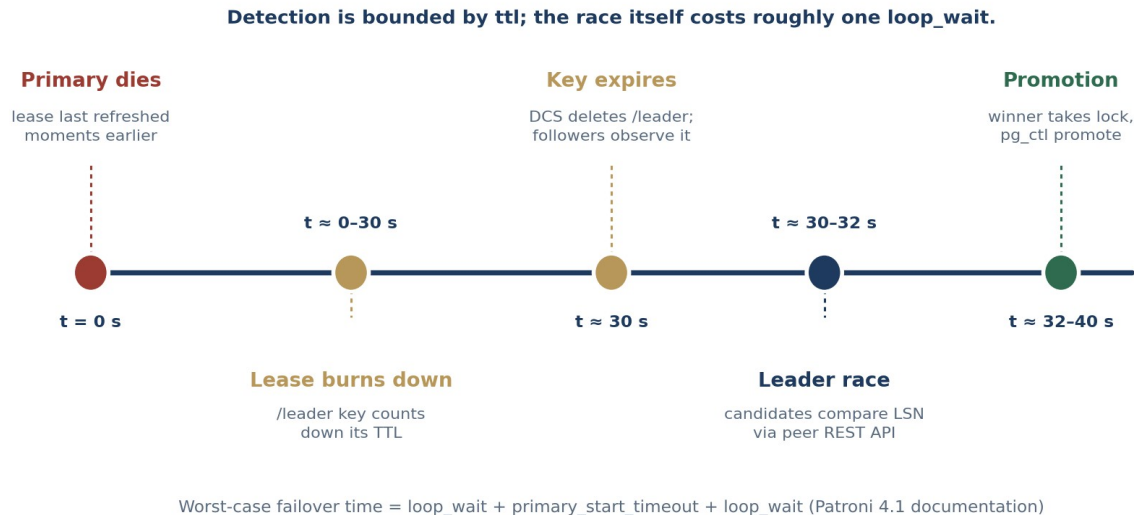


Figure 4 — Unplanned failover with default timing. Detection is bounded by ttl; the race itself costs roughly one loop_wait.

At the moment of failure nothing happens, and this surprises operators. The lease was refreshed shortly before the crash and remains valid for the remainder of its term. The standbys observe a leader key that is present and unexpired, and correctly continue following a primary that no longer exists. Nothing is wrong; the system is waiting for the only evidence it trusts.

Once the lease expires, the DCS removes `/leader`. On their next iteration the standbys observe its absence and begin the race. Each candidate queries its peers' REST APIs for their WAL positions, and a node that finds a peer ahead of it defers. The node that believes itself furthest ahead attempts the compare-and-create on `/leader`; the store admits exactly one. The winner promotes PostgreSQL, which switches to a new timeline, and writes the transition into `/history`.

7.2 Eligibility: who may compete

Before a node may join the race it must satisfy every one of the following. The list is worth memorising, because "why did nothing get promoted?" is answered by working down it.

Test	Failing it means	Controlled by
Reachable via its own REST API	The node cannot be interrogated by peers and is excluded	Network, restapi configuration
nofailover tag is not set	The node is administratively barred from promotion	tags.nofailover / failover_priority
Watchdog functional, if required	Fencing cannot be guaranteed, so promotion is refused	watchdog.mode: required
Lag within maximum_lag_on_failover	The node is too far behind; promoting it would lose more than the permitted bound	maximum_lag_on_failover
Timeline not behind the cluster	The node diverged and would promote a stale history	check_timeline (default false)
Listed in <code>/sync</code> , in synchronous mode	The node is not guaranteed to hold every acknowledged commit	synchronous_mode

THE STALL THAT LOOKS LIKE A BUG

If every surviving standby exceeds `maximum_lag_on_failover`, no automatic promotion occurs and the cluster stays read-only until an operator intervenes. This is correct behaviour, not a defect: Patroni is declining to lose more data than you told it to tolerate. The operator's choice at that point is to accept the loss with an explicit `patronictl failover`, or to recover the old primary.

7.3 Failover and switchover are different operations

	Failover	Switchover
Intent	React to a failure that has already happened	Move the primary deliberately, with no failure present
Requires a leader to exist	No	Yes
Requires a named candidate	Yes	No — any eligible node may win the race
Can be scheduled	No	Yes, unless paused
Permitted while paused	Yes	Yes, but only to a specific candidate
Data loss risk	Real — eligibility checks are relaxed in a leaderless cluster	None in the ordinary case; the old primary is shut down cleanly first

The operational rule follows without argument: for anything planned — kernel patching, hardware maintenance, a rebalance across zones — use a switchover. A manual failover in a healthy cluster forfeits the clean shutdown and the guarantee that comes with it.

```
# Planned maintenance on the current primary
patronictl -c /etc/patroni/patroni.yml list
patronictl -c /etc/patroni/patroni.yml switchover --candidate node-2

# ... patch and reboot the old primary, then confirm it rejoined
patronictl -c /etc/patroni/patroni.yml list
```

7.4 Shaping elections with tags

Tags are per-node attributes set in the local configuration file. They are how you express topology intent — that a node in a remote datacentre should never be promoted automatically, or that a reporting replica should never receive load-balanced traffic.

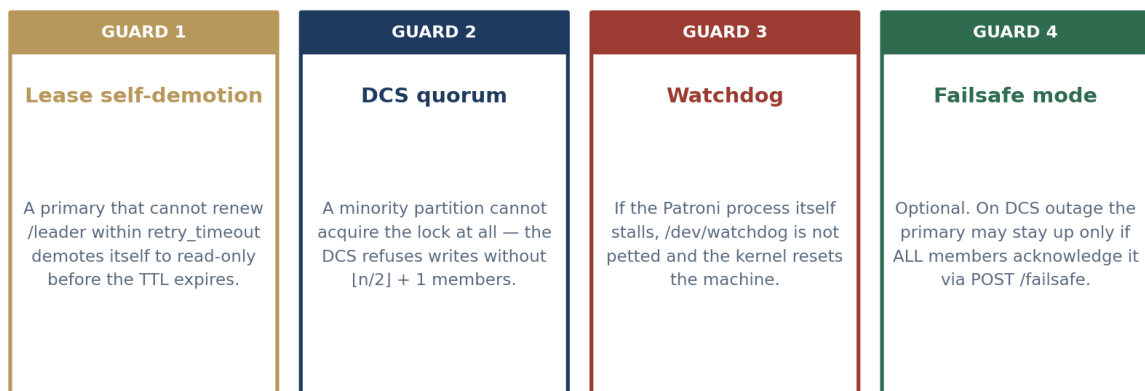
Tag	Effect
<code>nofailover</code>	Node may never win the leader race. Equivalent to <code>failover_priority: 0</code> .
<code>failover_priority</code>	Integer preference among otherwise equal candidates; higher wins. Set this or <code>nofailover</code> , never both.
<code>nosync</code>	Node is never chosen as a synchronous standby.
<code>sync_priority</code>	Preference during synchronous standby selection; 0 or negative behaves like <code>nosync</code> .
<code>noloadbalance</code>	Node returns 503 from GET /replica, removing it from proxy read pools.

Tag	Effect
clonefrom	Prefer this node as the source when building a new replica, sparing the primary the I/O.
replicatefrom	Names another replica to stream from, producing cascading replication.
nostream	Node does not stream at all; it recovers from the archive via restore_command.

8. Preventing Split-Brain

Chapter 1 argued that split-brain is the failure worth spending everything to avoid. This chapter enumerates precisely what Patroni spends. Figure 5 shows the four guards, which are independent and layered rather than alternatives.

Four independent guards against two primaries



Guards 1 and 2 are always active. Guard 3 requires a watchdog device; Guard 4 must be enabled in /config.

Figure 5 — The four guards. The first two are always active; the third requires a watchdog device and the fourth must be enabled explicitly.

8.1 Guard one — lease self-demotion

The primary is responsible for its own demotion. If it cannot refresh /leader within retry_timeout, it demotes itself to read-only before the lease can expire — that is, before any other node is permitted to promote. The critical property is that this requires no external actor. Nobody has to notice; the node fences itself.

8.2 Guard two — DCS quorum

A node in a minority partition cannot acquire the lock at all, because the DCS in that partition cannot serve writes without a majority. This is what makes distributing the DCS across failure domains a correctness decision rather than an availability preference: it determines which side of a partition retains the ability to elect anything.

8.3 Guard three — the watchdog

Guards one and two both assume the agent is running. The watchdog covers the case where it is not — hung, suspended, or starved — by moving enforcement into the kernel, as described in Section 3.6. Without it, a frozen agent leaves a writable PostgreSQL behind a lock that has already been given to somebody else, which is the exact configuration of a split-brain.

8.4 Guard four — DCS failsafe mode

The first three guards produce an uncomfortable consequence: if the DCS becomes entirely unavailable while every PostgreSQL node is healthy, the primary demotes itself and the cluster goes read-only. A dependency outage has caused a database outage even though no database failed. On Kubernetes, where the API server is shared with everything else in the cluster, this is common enough to be a real operational problem.

Failsafe mode addresses it. When enabled in `/config`, a leader that cannot update the lock for reasons other than a value mismatch may continue as primary — but only if it can reach every member listed in `/failsafe` via `POST /failsafe` and every one of them acknowledges. Replicas treat an incoming failsafe request as evidence the primary is alive and decline to start a leader race, caching that fact for `ttr` seconds.

WHY ALL MEMBERS, AND NOT A QUORUM

Patroni's own documentation is explicit on this point. Patroni node placement is not constrained the way DCS member placement is, so a majority of Patroni nodes may sit on the losing side of a partition together with the primary while a minority sits on the winning side. Requiring **unanimous** acknowledgement is the only check that reliably detects that arrangement. If a single member is unreachable — including one that was simply terminated — the primary demotes.

```
# Enable failsafe mode (verify all members run a current Patroni first)
patronictl -c /etc/patroni/patroni.yml edit-config -s failsafe_mode=true

# Confirm; an active failsafe reports failsafe_mode_is_active
curl -s http://localhost:8008/patroni | jq '.failsafe_mode_is_active'
```

9. Replication Modes and the Durability Spectrum

Patroni does not implement replication; it configures PostgreSQL's. What it adds is the guarantee that the configuration remains coherent as membership changes, and that promotion respects whatever durability contract you asked for. Figure 6 places the four modes on the availability–durability spectrum.

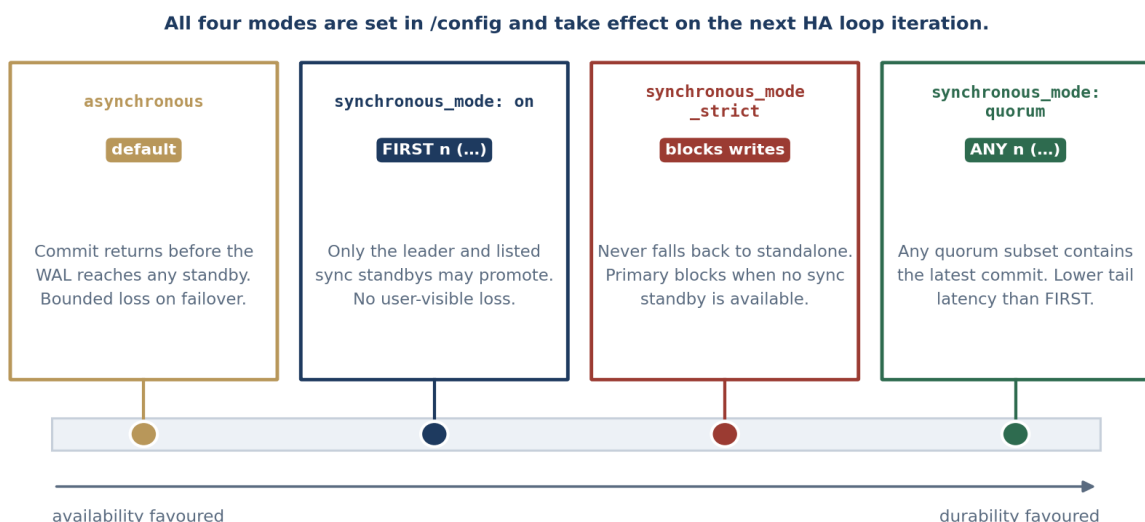


Figure 6 — The four replication modes. All are set in `/config` and take effect on the next loop iteration.

9.1 Asynchronous — the default, and its bound

By default a commit returns as soon as the primary has flushed locally. If the primary is lost, transactions not yet replicated are stranded on a forked timeline and are, for practical purposes, unrecoverable.

The loss is bounded rather than unlimited, and the bound is worth stating precisely. Because the primary's WAL position is sampled rather than observed continuously, worst-case loss is `maximum_lag_on_failover` bytes plus whatever was written during the last `tli` seconds — averaging `loop_wait/2` in practice. Steady-state replication delay on healthy infrastructure is normally well under a second, so the typical loss is far smaller than the worst case. But it is not zero, and asynchronous mode should never be described to a business stakeholder as though it were.

9.2 Synchronous mode

Setting `synchronous_mode: on` changes the promotion rule, and this is the part frequently missed. It is not merely that commits wait for a standby; it is that Patroni maintains the current synchronous set in `/sync` and permits only the leader and the nodes listed there to be promoted automatically. A standby that was never synchronous cannot become primary, because it cannot be shown to hold every acknowledged commit.

```
-- /config
synchronous_mode: on
synchronous_node_count: 2

-- /sync as maintained by Patroni
{ "leader": "node0", "sync_standby": "node1,node2" }

-- resulting postgresql.conf
synchronous_standby_names = 'FIRST 2 (node1,node2)'
```

Ordinary synchronous mode preserves availability at the edge: if no suitable standby remains, the primary falls back to standalone operation and continues accepting writes without replication guarantees. When the primary then fails, nothing is promoted. Patroni is choosing to stall rather than to promote a node that might be missing acknowledged commits.

9.3 Strict mode

`synchronous_mode_strict` removes that fallback. The primary never reverts to standalone, and blocks writes until a synchronous standby is available. This is the only configuration that comes close to guaranteeing every commit exists on two nodes, and it converts the loss of your last standby from a durability downgrade into a write outage.

EVEN STRICT MODE IS NOT ABSOLUTE

The Patroni documentation records an edge case arising from how PostgreSQL implements synchronous replication: if a backend is cancelled while waiting for replication acknowledgement — because a client timed out, or the backend failed — its changes become visible to other backends before they have been replicated, and can be lost if a standby is then promoted. Strict mode narrows the window dramatically; it does not close it.

9.4 Quorum commit

Setting `synchronous_mode: quorum` uses PostgreSQL's `ANY` form instead of `FIRST`. Rather than naming which standbys must acknowledge, it requires that some number of them do, and Patroni records in `/sync` both the voting set and the quorum size needed to reason about it afterwards.

```
-- /config
synchronous_mode: quorum
synchronous_node_count: 2

-- /sync
{ "leader": "node0", "sync_standby": "node1,node2,node3", "quorum": 1 }

-- resulting postgresql.conf
synchronous_standby_names = 'ANY 2 (node1,node2,node3)'
```

The benefit is latency. Under FIRST, commit latency is hostage to the specific named standbys, so one slow replica slows every transaction. Under ANY, a slow replica is simply outvoted by faster peers, which compresses the tail. The recorded quorum size is what makes promotion safe afterwards: to establish that a candidate holds the latest commit, Patroni compares its LSN against at least quorum other voters, and any subset achieving quorum is guaranteed to contain a node with the newest commit.

10. Bootstrap, Replica Creation and Rejoin

10.1 The bootstrap race

On first start, every agent finds an empty namespace and every agent tries to initialise. The `/initialize` key resolves it: each attempts an atomic create, exactly one wins, runs `initdb` and any post-bootstrap script, and writes the resulting PostgreSQL system identifier into the key. The losers observe that the key now exists and switch to creating themselves as replicas of the winner.

The system identifier stored there is a durable safety check. A node whose data directory carries a different identifier is not a member of this cluster — it is a different database that happens to share a name — and Patroni refuses to let it join.

10.2 Creating a replica

A node needing data uses a replica creation method, `pg_basebackup` by default. The default is safe and simple but streams the entire database from a source node; on a multi-terabyte cluster that is hours of load. Two mitigations exist and both are worth knowing: the `clonefrom` tag directs new replicas to build from a nominated standby rather than the primary, keeping the I/O off the node serving writes; and a custom method can restore from your existing backup infrastructure instead, which for `pgBackRest` or `WAL-G` users is dramatically faster and puts no load on the running cluster at all.

10.3 Rejoining a former primary

When a demoted primary returns, it usually cannot simply start streaming. It may have written WAL that the new primary never received, putting it on a diverged timeline. Reconciling this by rebuilding from scratch is correct but expensive.

`pg_rewind` is the cheap path: it rewinds only the blocks that differ since the divergence point, turning hours into minutes. It has a hard prerequisite that must be satisfied at cluster creation time, not at the moment you need it.

DECIDE THIS BEFORE YOU BUILD THE CLUSTER

`pg_rewind` requires either that the cluster was initialised with data page checksums (`--data-checksums` passed to `initdb`) or that `wal_log_hints` is on. Neither can be enabled later without effort — checksums

require a rebuild or an offline `pg_checksums` run, and `wal_log_hints` needs a restart. Set both at bootstrap. The cost is a small write amplification; the benefit is that recovering a failed primary is a short operation instead of a full clone.

When rewind is impossible or fails, the fallback is a full rebuild, which discards the local data directory and creates the replica afresh:

```
patronictl -c /etc/patroni/patroni.yml reinit <scope> <member>

# When every replica has failed, build directly from the leader
curl -s -XPOST -d '{"from-leader":true}' http://localhost:8008/reinitialize
```

10.4 Timelines and history

Every promotion increments the PostgreSQL timeline, which is how the system distinguishes "WAL position 0/5000000 on the history that existed before the failover" from the same position on the history after it. Patroni mirrors this into `/history`, giving you a queryable record of every promotion the cluster has ever performed, with timestamps.

By default Patroni does not consider timelines during elections. Setting `check_timeline: true` adds the test in Section 7.2, refusing to promote a node whose timeline is behind the cluster's. It is a reasonable hardening for clusters that have experienced divergence, at the cost of shrinking the eligible candidate set.

PART IV

Judgement: Practice, Limits and Alternatives

11. Best Practices

11.1 Architecture

- **Three PostgreSQL nodes minimum for automatic failover.** Two nodes cannot distinguish a peer failure from a partition. If two is genuinely the budget, run `synchronous_mode`, which the documentation notes makes two-node clusters workable — at the cost of write availability when the standby is gone.
- **An odd number of DCS members, distributed across the failure domains you intend to survive.** Three members in one rack give you quorum arithmetic and no actual protection.
- **Separate DCS hosts from PostgreSQL hosts where budget allows.** Co-location creates a correlated failure in which database load starves the store and triggers a self-inflicted failover.
- **Make the connection router redundant.** A single proxy in front of an HA cluster caps your availability at the proxy's.

11.2 Configuration

- **Enable the watchdog.** It is the only guard that survives the agent itself failing, and `softdog` costs nothing.
- **Initialise with `--data-checksums` and `wal_log_hints: on`.** This is the single highest-leverage decision at build time, and it cannot be revisited cheaply.
- **Use replication slots, but monitor them.** Slots stop the primary discarding WAL a replica still needs. An inactive slot will fill your WAL volume and stop the primary, which turns a redundancy feature into an outage.
- **Change dynamic configuration only through `patronictl edit-config` or the REST API.** Editing `postgresql.conf` by hand on one node produces a divergence Patroni will overwrite, usually at the least convenient moment.
- **Restart PostgreSQL with `patronictl restart`, never `systemctl restart patroni`.** Restarting the agent on a primary can trigger a failover; the documentation says so explicitly.

11.3 Operations

- **Pause before maintenance that would look like a failure.** `patronictl pause` keeps the agents reporting while stopping them from acting.
- **Switchover for anything planned.** Reserve failover for situations where there is no healthy leader to switch away from.
- **Rehearse failover on a schedule.** An HA cluster whose failover path has never been executed is an untested backup with better marketing.
- **Alert on the right signals:** `cluster_unlocked`, `pending_restart`, replication lag, `dcslastseen` age and replication slot growth. Do not alert on `etcd` Raft leader changes — they are routine and unrelated to your database.

12. Trade-offs and When Patroni Is the Wrong Tool

12.1 What adopting Patroni costs

Patroni buys automatic, safe failover. It is honest to state the price. You acquire two new components to operate — the agents and the DCS — and the DCS is now a dependency of your database's write availability. You give up the ability to manage PostgreSQL directly: starting the postmaster by hand is now a fault. You accept a failover window bounded by ttl rather than instantaneous cutover. And you take on genuine conceptual complexity, which is precisely why this book exists.

12.2 Failures Patroni does not address

Failure	Why Patroni does not help	What does
Data corruption	Corrupt blocks replicate faithfully to every standby	Backups with PITR; checksums for detection
Accidental DROP or bad migration	The statement is valid WAL and reaches all replicas in milliseconds	PITR; a delayed standby via <code>recovery_min_apply_delay</code>
Slow queries, bloat, bad plans	Entirely outside Patroni's concern	Query tuning, autovacuum configuration, monitoring
Loss of the whole site	A local cluster cannot survive its own datacentre	Standby cluster in a second region; cross-region backups
Application-side connection failure	Patroni does not touch client connections	Redundant router; retry and failover logic in the driver
Committed transactions lost in async mode	Bounded loss is the documented contract of the mode	<code>synchronous_mode</code> , or quorum commit

THE SENTENCE TO PUT IN YOUR RUNBOOK

Patroni is a high-availability tool, not a disaster-recovery tool and not a backup. It protects against the loss of a node. It offers no protection whatsoever against the loss of data, because replication is precisely the mechanism by which bad data reaches everywhere else. A Patroni cluster without tested backups is not a resilient system.

12.3 When to choose something else

- A single non-critical database where an hour of downtime is acceptable. Streaming replication plus a documented manual promotion is simpler and has fewer failure modes.
- A team with no appetite for operating a consensus store. A managed service — Amazon RDS, Aurora, Cloud SQL, Azure Database — moves this problem to a vendor. That is a legitimate engineering decision, not a defeat.
- An estate already standardised on Pacemaker and Corosync with in-house expertise. Adding a second HA philosophy for one database is usually a net loss.
- Kubernetes-native deployments. Patroni works there and underpins several operators, but the CloudNativePG or Zalando operators may fit the operational model better than raw Patroni.

13. Common Mistakes

16. **Treating a two-node cluster as highly available.** It is not, unless synchronous mode is on, and even then the availability characteristics differ from what most operators expect.
17. **Health-checking /health for the write pool.** A demoted primary passes it. Use /primary.
18. **Skipping the watchdog.** Every argument for skipping it is an argument that the agent will never hang, which is not a property anyone can guarantee.
19. **Building without checksums or wal_log_hints.** Discovered at the worst possible moment: when a failed primary needs to rejoin and only a full clone will do.
20. **Editing postgresql.conf directly.** Changes are silently overwritten, producing a puzzling class of bug where a setting reverts by itself.
21. **Restarting the agent to restart the database.** On a primary this can hand leadership to another node unintentionally.
22. **Deploying an even number of DCS members.** Four members tolerate the same single failure as three, while adding a machine that can fail.
23. **Ignoring replication slot growth.** An inactive slot silently retains WAL until the volume is full and the primary stops.
24. **Setting ttl below loop_wait + 2 × retry_timeout.** Produces failovers during ordinary jitter, which then get blamed on Patroni being unstable.
25. **Never testing failover.** The most common and the most expensive mistake in this list.

14. Comparative Analysis

Patroni is not the only answer to automatic PostgreSQL failover, and choosing it should be a decision rather than a default. The alternatives differ chiefly in where they place the arbitration authority.

Tool	Arbitration model	External dependency	Strengths	Principal limitation
Patroni	External DCS holds a lease; agents race for it	etcd, Consul, ZooKeeper or K8s API	Strong split-brain guarantees; multiple DCS choices; large deployed base; rich REST surface	Conceptual and operational complexity; DCS becomes a write-availability dependency
repmgr	repmgrd witness and voting among nodes	None beyond PostgreSQL	Simple to adopt; familiar to long-time PostgreSQL administrators; low operational weight	Weaker partition guarantees; fencing is left to the operator
pg_auto_failover	A dedicated monitor node holds the state machine	One monitor (itself a PostgreSQL instance)	Very simple mental model; purpose-built for two-node plus monitor	The monitor is a single point of arbitration; less flexible topology
PAF on Pacemaker	Corosync cluster membership with fencing agents	Pacemaker, Corosync, STONITH devices	Genuine hardware fencing; integrates with non-database resources	Steep learning curve; heavy stack for one database
Managed cloud services	Proprietary, operated by the vendor	The vendor	No HA machinery to operate; failover is an SLA line item	No visibility or control over failover semantics; cost; migration lock-in

14.1 Conclusion

For a self-managed PostgreSQL estate of meaningful size, Patroni is the reasonable default, and the reason is narrow rather than general. Its correctness argument is the strongest available outside hardware fencing, because it delegates the hard part to a consensus store built for that purpose instead of implementing voting itself. That single design decision is what earns it the complexity it charges you.

The honest caveats are equally narrow. If your estate is one small database, Patroni is overbuilt and manual promotion is defensible. If you already operate Pacemaker competently, PAF gives you real fencing and one stack instead of two. If you have no appetite for a consensus store, a managed service is a legitimate answer rather than an admission. And if you are already inside Kubernetes, an operator built on Patroni will likely serve you better than Patroni configured by hand.

What is not defensible is choosing Patroni and then declining to understand it. Every guarantee in this book is conditional on configuration, and a cluster with the watchdog disabled, an even DCS count and untested failover has the complexity without the guarantees.

15. Advanced Extensions

15.1 Standby clusters

A standby cluster is a complete Patroni cluster whose leader — the standby leader — replicates from a primary in another cluster instead of holding a writable database. It has its own DCS namespace and runs its own elections, so losing the standby leader triggers a local election rather than a cross-region failover. This is the supported building block for cross-region disaster recovery, and promotion between regions remains a deliberate operator action.

15.2 Kubernetes

Patroni can use the Kubernetes API itself as the DCS, storing state in annotations on Endpoints or ConfigMaps and eliminating a separate store. The liveness and readiness endpoints of Section 3.5 exist for this environment: readiness ensures a freshly started pod only receives traffic once it has caught up, which combined with a PodDisruptionBudget prevents a rolling node upgrade from terminating the leader prematurely. The API server's shared nature is also the origin of failsafe mode.

15.3 Citus

Patroni has first-class support for Citus, managing high availability for both the coordinator and the worker groups, and maintaining the node metadata that Citus needs as membership changes. Patroni keeps the list of replicas with the appropriate role and state — excluding those tagged `noloadbalance` — registered for distributed query routing.

15.4 Cascading and delayed replicas

The `replicatefrom` tag builds cascading topologies in which a replica streams from another replica, reducing the number of WAL senders on the primary — useful when a remote site needs several standbys but only one wide-area link. Separately, `recovery_min_apply_delay` creates a deliberately lagging standby, which is one of the few defences against the accidental-DROP scenario in Section 12.2: the delayed node still holds the data for the length of the delay window.

PART V

Reinforcement

16. Laboratory Exercises

These exercises are designed for a disposable three-node cluster — containers, virtual machines or a Vagrant environment — running PostgreSQL with etcd as the DCS. None of them should be performed against production. Each states its objective, the theory it verifies, and the observation that constitutes success.

BEFORE YOU BEGIN

Confirm you are on a lab cluster by checking the scope name before every destructive step. Labs 3, 4 and 5 deliberately break the cluster. Substitute your own scope, member names and configuration path throughout.

Lab 1 — Foundational: read the cluster's mind

Objective. Establish that the DCS, not any database, holds the authoritative cluster state.

Theory verified. Chapter 4: the key hierarchy and which keys carry a TTL.

```
# The cluster as Patroni presents it
patronictl -c /etc/patroni/patroni.yml list

# The same truth, read directly from etcd
etcdctl get --prefix /service/<scope>/ --keys-only
etcdctl get /service/<scope>/leader
etcdctl get /service/<scope>/config

# Watch the lease being renewed – the TTL resets every loop_wait
watch -n 1 'etcdctl lease list'
```

Expected observation. The value of /leader is the name of the node that patronictl reports as Leader. Every key you listed corresponds to a row in the table in Section 4.1.

Questions. Which keys did you find that carry a lease, and which do not? Why would giving /config a TTL be a serious design error?

Lab 2 — Foundational: the REST API as a routing decision

Objective. Demonstrate that role, not liveness, is what a proxy must check.

Theory verified. Section 3.5 and the routing rule that accompanies it.

```
# On the primary, then repeat every line on a replica
curl -s -o /dev/null -w 'primary %s\n' localhost:8008/primary
curl -s -o /dev/null -w 'replica %s\n' localhost:8008/replica
curl -s -o /dev/null -w 'health %s\n' localhost:8008/health
curl -s -o /dev/null -w 'lagged %s\n' 'localhost:8008/replica?lag=1MB'
```

```
curl -s localhost:8008/cluster | jq '.members[] | {name, role, state, lag}'
```

Expected observation. Every healthy node returns 200 from /health regardless of role. Only one returns 200 from /primary. This is the whole argument for routing writes at /primary.

Lab 3 — Applied: observe a controlled switchover

Objective. Watch the lock change hands cleanly and confirm the timeline advanced.

Theory verified. Sections 7.1 and 7.3, and the /history key.

```
# Terminal 1 – follow the leader's reasoning
journalctl -u patroni -f

# Terminal 2 – perform the switchover
patronictl -c /etc/patroni/patroni.yml switchover --candidate <member>

# Terminal 3 – confirm the new timeline was recorded
curl -s localhost:8008/history | jq .
psql -c 'SELECT timeline_id FROM pg_control_checkpoint();'
```

Expected observation. The old primary shuts down cleanly before the new one promotes, and /history gains an entry. Contrast the log narrative with Lab 4, where no clean shutdown occurs.

Lab 4 — Applied: measure a real failover window

Objective. Verify that detection time is governed by ttl, and measure it.

Theory verified. Chapter 5 lease arithmetic and Figure 4.

```
# Record the current timing so you can reason about the result
patronictl -c /etc/patroni/patroni.yml show-config | grep -E 'ttl|loop_wait|retry_timeout'

# Kill the primary abruptly – no clean shutdown, no lock release
sudo systemctl stop --signal=SIGKILL patroni # on the primary node

# From a surviving node, time the promotion
time until curl -sf localhost:8008/primary >/dev/null; do sleep 1; done
```

Expected observation. Nothing happens for most of the window; promotion follows shortly after the lease expires. Repeat with ttl lowered to 20 and loop_wait to 5, and confirm the window shrinks proportionally.

Questions. Why did the standbys wait rather than promoting immediately when the primary stopped responding? What would break if you set ttl to 15 while leaving retry_timeout at 10?

Lab 5 — Advanced: partition the DCS and observe both guards

Objective. Demonstrate lease self-demotion, then show failsafe mode changing the outcome.

Theory verified. Sections 8.1 and 8.4.

```
# Part A – failsafe disabled. Cut the primary off from the DCS only.
sudo iptables -A OUTPUT -p tcp --dport 2379 -j DROP
```

```
# Observe on the primary: the lease cannot be refreshed
journalctl -u patroni -f
psql -c 'SELECT pg_is_in_recovery();' # becomes true after demotion

sudo iptables -D OUTPUT -p tcp --dport 2379 -j DROP

# Part B – enable failsafe and repeat exactly the same partition
patronictl -c /etc/patroni/patroni.yml edit-config -s failsafe_mode=true
sudo iptables -A OUTPUT -p tcp --dport 2379 -j DROP
curl -s localhost:8008/patroni | jq '.failsafe_mode_is_active'

sudo iptables -D OUTPUT -p tcp --dport 2379 -j DROP
```

Expected observation. In Part A the primary demotes itself and the cluster goes read-only although no database failed. In Part B it stays primary, having reached every member over the REST API. Now stop one replica entirely and repeat Part B: the primary demotes again, because failsafe requires unanimous acknowledgement.

Questions. Why is unanimity required rather than a quorum? What operational risk have you accepted by enabling failsafe mode?

17. Assessment

Twenty questions covering the full arc of the book. The answer key follows. A score below fourteen suggests re-reading Parts II and III before operating a production cluster.

Section A — Concepts and components (1-7)

- Q1. State the three responsibilities of a Patroni agent.
- Q2. Explain why a standby cannot safely decide on its own that the primary has died.
- Q3. Which components sit in the data plane, and which in the control plane?
- Q4. Name the four primitives Patroni requires from any Distributed Configuration Store.
- Q5. Which two keys in the cluster namespace carry a time-to-live, and why only those two?
- Q6. What does the /initialize key contain, and what mistake does it prevent?
- Q7. Why is an even number of DCS members strictly worse than the odd number below it?

Section B — Behaviour and timing (8-14)

- Q8. List the five stages of one heartbeat-loop iteration in order.
- Q9. State the constraint relating ttl, loop_wait and retry_timeout, and explain the reasoning behind it.
- Q10. A primary loses power at t = 0 with default timing. Describe what happens at t = 5 s and at t = 35 s.
- Q11. Name four tests a node must pass to be eligible for automatic promotion.
- Q12. Every surviving replica exceeds maximum_lag_on_failover. What does Patroni do, and why is that correct?
- Q13. Give three differences between a failover and a switchover.
- Q14. Why is the heartbeat loop described as idempotent, and what operational property follows from it?

Section C — Guarantees and judgement (15–20)

- Q15.** Name the four guards against split-brain and state which are active by default.
- Q16.** Why must failsafe mode require acknowledgement from every member rather than a majority?
- Q17.** Distinguish `synchronous_mode` from `synchronous_mode_strict` in terms of what happens when the last standby is lost.
- Q18.** What advantage does quorum commit offer over FIRST-style synchronous replication?
- Q19.** Which two settings must be chosen at bootstrap to keep `pg_rewind` available, and what is the consequence of omitting them?
- Q20.** An engineer proposes that Patroni removes the need for backups. Rebut this in two sentences.

Answer Key

Section A

- **A1.** Manage the local PostgreSQL lifecycle; publish and consume cluster state through the DCS; contend for, hold and surrender the leader lock.
- **A2.** Because it cannot distinguish a crashed primary from a slow one or from a network partition — perfect failure detection is impossible in an asynchronous network. Acting on that ambiguity unilaterally produces split-brain.
- **A3.** Data plane: PostgreSQL and its streaming replication. Control plane: the Patroni agents, the DCS, the REST API, `patronictl` and the routing decisions of the proxy.
- **A4.** Atomic create, compare-and-swap update, automatic expiry via TTL, and linearizable reads.
- **A5.** `/leader` and `/members/<name>`. Both must vanish by themselves when the writer dies, which is exactly what makes failure detectable without an observer. Configuration and history must survive the death of any node and therefore cannot expire.
- **A6.** The PostgreSQL system identifier. It prevents a second node initialising over an existing cluster, and prevents a foreign data directory joining a cluster it does not belong to.
- **A7.** Four members tolerate the same single failure as three — quorum is three either way — while adding another machine that can itself fail, so the probability of losing quorum rises.

Section B

- **A8.** Observe self; read the DCS; publish own state to the member key; decide and act on one branch; sleep for `loop_wait`.
- **A9.** $\text{ttl} \geq \text{loop_wait} + 2 \times \text{retry_timeout}$. The leader must have at least one complete opportunity to refresh the lease, including a full round of retries, before it expires — otherwise ordinary jitter causes failover.
- **A10.** At $t = 5 \text{ s}$ nothing has happened: the lease is still valid and the standbys are correctly following an absent primary. At $t \approx 35 \text{ s}$ the lease has expired, the key has been removed, and the standbys have begun the leader race.
- **A11.** Any four of: reachable via its REST API; `nofailover` not set; watchdog functional if required; lag within `maximum_lag_on_failover`; timeline not behind the cluster when `check_timeline` is true; listed in `/sync` when synchronous mode is on.
- **A12.** Nothing is promoted and the cluster stays read-only. This is correct because promoting would discard more transactions than the configured bound permits; the decision to accept that loss belongs to an operator, not to the software.

- **A13.** Any three of: failover needs no existing leader, switchover does; failover requires a named candidate, switchover does not; switchover can be scheduled, failover cannot; switchover shuts the old primary down cleanly, failover does not; failover relaxes eligibility checks in a leaderless cluster and can lose data.
- **A14.** Because each iteration re-derives the correct action from current DCS state rather than carrying a decision forward. It follows that an agent can be killed and restarted at any point with no recovery procedure and no local state to reconcile.

Section C

- **A15.** Lease self-demotion, DCS quorum, the watchdog, and failsafe mode. The first two are always active; the watchdog requires a device to be configured and failsafe mode must be enabled in /config.
- **A16.** Because Patroni node placement is unconstrained, a majority of Patroni nodes may sit on the losing side of a partition along with the primary. Only checking that every member is reachable reliably detects that arrangement.
- **A17.** In synchronous_mode the primary falls back to standalone operation and keeps accepting writes without a replication guarantee. In strict mode it never falls back and instead blocks writes until a synchronous standby returns — trading write availability for durability.
- **A18.** Lower tail latency. Under ANY, a single slow standby is outvoted by faster peers rather than delaying every commit, and the recorded quorum size still allows Patroni to prove a candidate holds the latest commit.
- **A19.** Initialise with --data-checksums and set wal_log_hints: on. Without at least one of them pg_rewind cannot run, so a former primary that diverged must be rebuilt by a full clone rather than a short rewind.
- **A20.** Patroni protects against the loss of a node, not the loss of data — corruption and mistaken DDL replicate faithfully to every standby within milliseconds. Recovery from those events requires point-in-time restore from backup, which Patroni neither provides nor replaces.

Appendix A — Parameter Quick Reference

Defaults as documented for Patroni 4.1.5. Confirm against your own deployment with `patronictl show-config` before relying on any value.

Timing and election

Parameter	Default	Notes
<code>ttl</code>	30 s	Leader lock lifetime; minimum 20
<code>loop_wait</code>	10 s	Heartbeat interval; minimum 1
<code>retry_timeout</code>	10 s	DCS/PostgreSQL retry window; minimum 3
<code>maximum_lag_on_failover</code>	1048576 bytes	Eligibility bound for automatic promotion
<code>primary_start_timeout</code>	300 s	Grace period for a crashed primary to recover in place; 0 fails over immediately
<code>check_timeline</code>	false	Refuse to promote a node behind the cluster timeline
<code>max_timelines_history</code>	0	0 keeps full history in the DCS
<code>failsafe_mode</code>	false	Permit the primary to survive a DCS outage if all members acknowledge

Replication and durability

Parameter	Default	Notes
<code>synchronous_mode</code>	off	off on quorum
<code>synchronous_mode_strict</code>	false	Never fall back to standalone operation
<code>synchronous_node_count</code>	1	Number of synchronous standbys Patroni maintains
<code>maximum_lag_on_syncnode</code>	-1	Disabled at 0 or below; controls swapping a lagging sync standby
<code>member_slots_ttl</code>	30 min	Retention of member replication slots after a member disappears; PostgreSQL 11+
<code>postgresql.use_slots</code>	true	Use replication slots for members
<code>postgresql.use_pg_rewind</code>	—	Requires data checksums or <code>wal_log_hints</code>

Frequently used commands

```
# Inspect
patronictl -c /etc/patroni/patroni.yml list
patronictl -c /etc/patroni/patroni.yml show-config
patronictl -c /etc/patroni/patroni.yml history
curl -s localhost:8008/cluster | jq .
curl -s localhost:8008/patroni | jq .

# Change state
```

```
patronictl -c /etc/patroni/patroni.yml switchover --candidate <member>
patronictl -c /etc/patroni/patroni.yml failover --candidate <member>
patronictl -c /etc/patroni/patroni.yml restart <scope> <member>
patronictl -c /etc/patroni/patroni.yml reinit <scope> <member>

# Maintenance
patronictl -c /etc/patroni/patroni.yml pause
patronictl -c /etc/patroni/patroni.yml resume
patronictl -c /etc/patroni/patroni.yml edit-config

# Configuration lifecycle
patroni --generate-sample-config /etc/patroni/patroni.yml
patroni --validate-config /etc/patroni/patroni.yml
```

Appendix B — References

References follow American Psychological Association (APA) 7th edition style. All sources were consulted and verified during the preparation of this volume.

etcd Authors. (2026). etcd documentation: Learning — Data model and API guarantees.

<https://etcd.io/docs/>

HashiCorp. (2026). Consul documentation: Sessions and KV store.

<https://developer.hashicorp.com/consul/docs>

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. In Proceedings of the 2014 USENIX Annual Technical Conference (pp. 305–319). USENIX Association.

<https://raft.github.io/raft.pdf>

Patroni Contributors. (2026). DCS failsafe mode (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/dcs_failsafe_mode.html

Patroni Contributors. (2026). Dynamic configuration settings (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/dynamic_configuration.html

Patroni Contributors. (2026). Patroni configuration (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/patroni_configuration.html

Patroni Contributors. (2026). Patroni REST API (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/rest_api.html

Patroni Contributors. (2026). Replication modes (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/replication_modes.html

Patroni Contributors. (2026). Watchdog support (Version 4.1.5) [Documentation].

<https://patroni.readthedocs.io/en/latest/watchdog.html>

Patroni Contributors. (2026). YAML configuration settings (Version 4.1.5) [Documentation].

https://patroni.readthedocs.io/en/latest/yaml_configuration.html

PostgreSQL Global Development Group. (2026). PostgreSQL 17 documentation: High availability, load balancing, and replication. <https://www.postgresql.org/docs/17/high-availability.html>

PostgreSQL Global Development Group. (2026). PostgreSQL 17 documentation: pg_rewind.

<https://www.postgresql.org/docs/17/app-pgrewind.html>